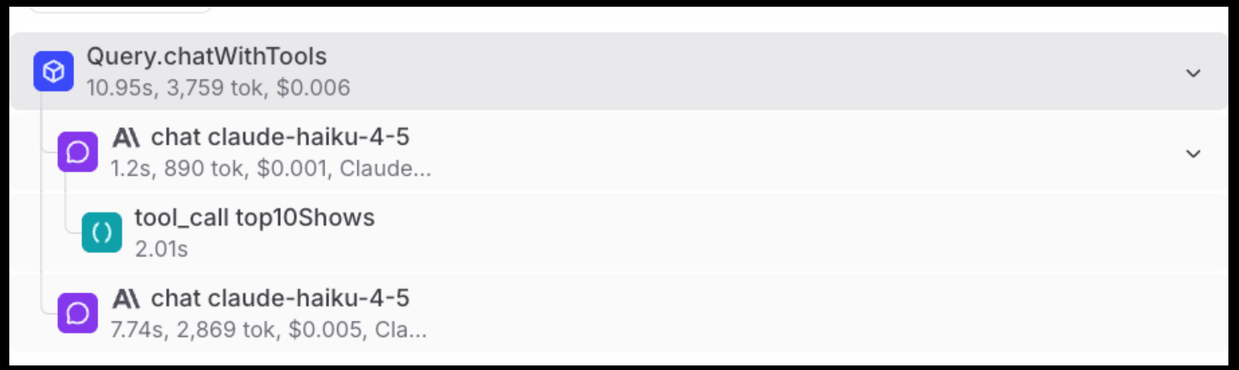
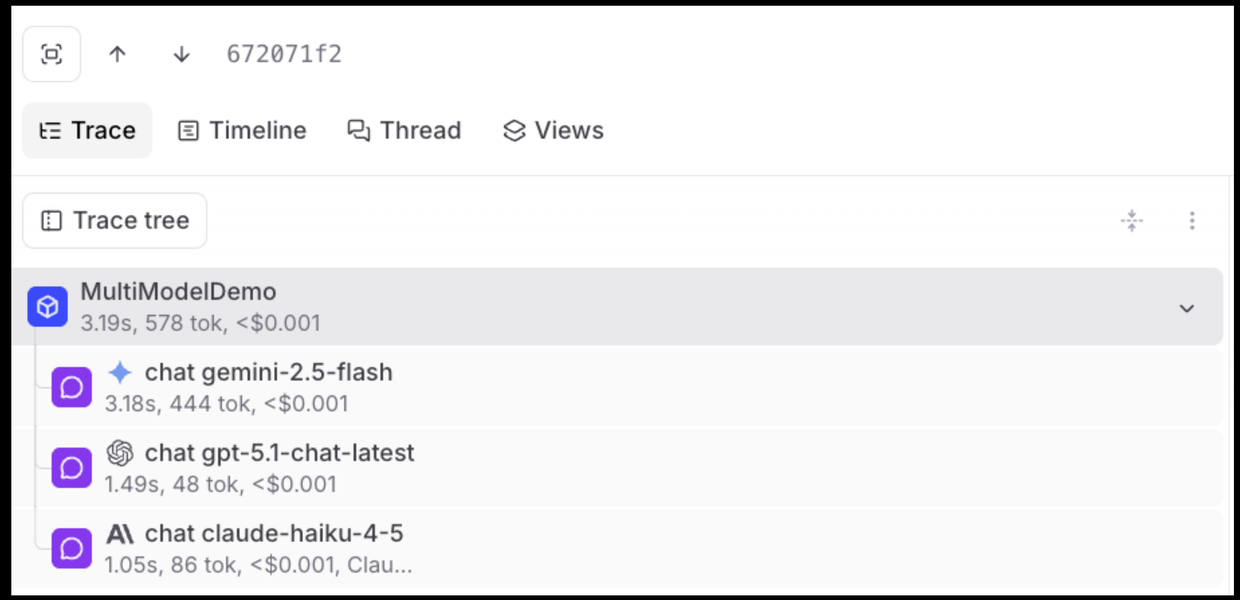
Braintrust tracing integration

```java
// Name the root span (optional — defaults to the DGS data fetcher name)
tracer.currentSpan().name("MultiModelDemo synchronous");

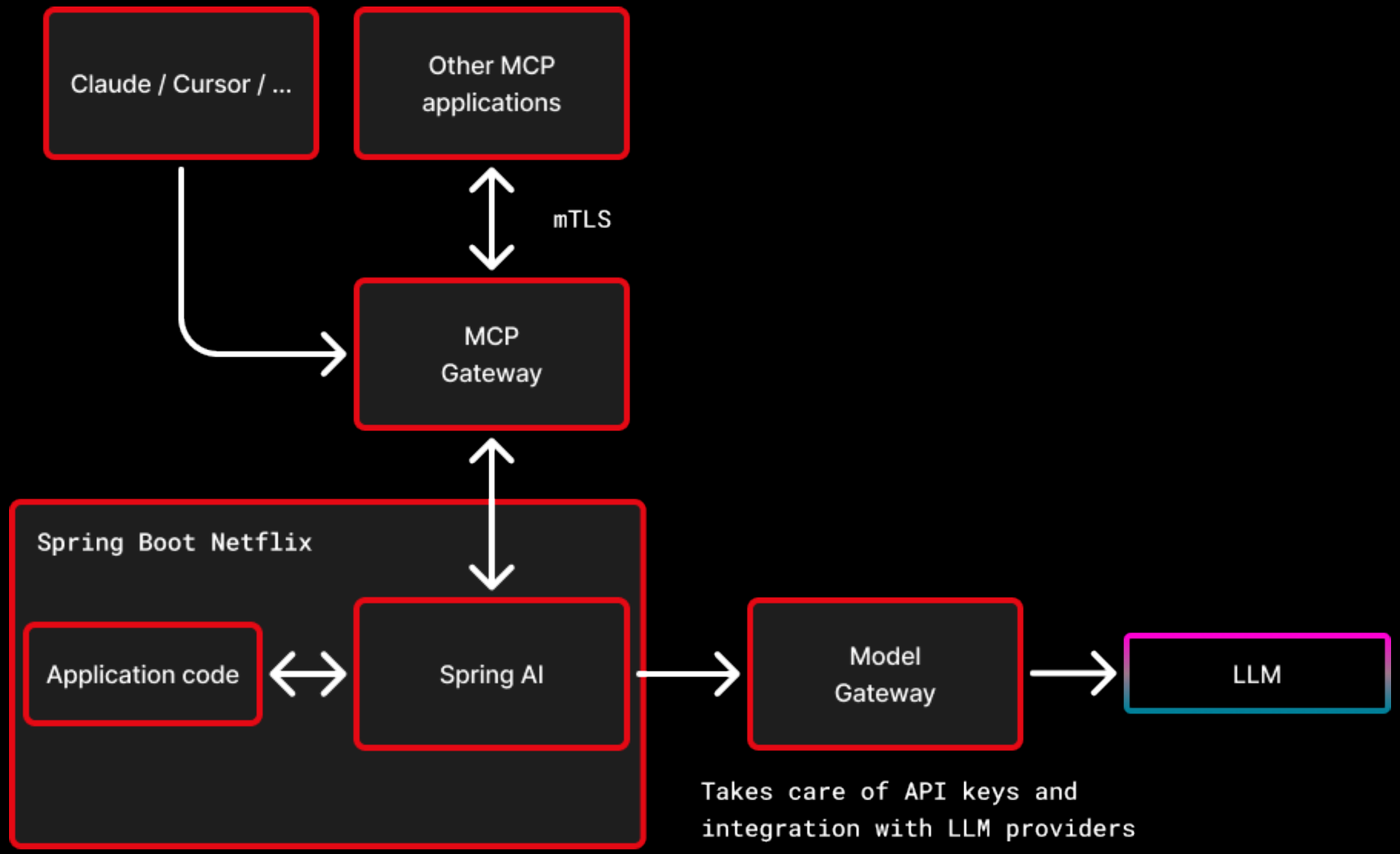
// Request-level metadata (appears on the task span)
tracer.currentSpan().tag("braintrust.metadata.agentic-flow", "multimodel");

var anthropicAnswer = anthropicClient.prompt(question)
        .advisors( AdvisorSpec spec ->
            spec.param( k: "braintrust.metadata.anotherParam",  v: "hello"))
        .call().content();
```





MCP



Takes care of API keys and integration with LLM providers

MCP

```java
@McpTool(name = "top10Shows", description = "Use to get a top 10 list of shows in a given genre")
public List<Show> top10(@McpArg(name = "genre", description = "Genre of the shows in the top 10 list") String genre) {

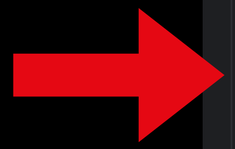
    LOGGER.info("Reading shows from JSON for MCP invocation");

    var objectMapper = new ObjectMapper();
    try {
        var shows = objectMapper.readValue(resource.getContentAsByteArray(), new TypeReference<List<Show>>() {
        });

        return shows.stream().filter(Show s -> s.genre().stream().anyMatch(String g -> g.equalsIgnoreCase(genre))).toList();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

Testing agents

Test slice to auto configure Spring AI + integrations



```java
@SpringBootTest(classes = {ChatWithoutRag.class})
@EnableAiTest
class ChatWithoutRagTest {

    @Autowired
    ChatWithoutRag chatWithoutRag;

    @Test
    void promptWithoutRagShouldKnowSpringBoot() {
        var response = chatWithoutRag.promptWithoutRag( question: "What is Spring Boot?");

        assertThat(response).isNotEmpty();
        assertThat(response).contains("Java", "framework");
    }
```




LLM as a judge

```java
@SpringBootTest(classes = {ChatWithoutRag.class})
@EnableAiTest
class ChatWithoutRagTest {

    @Autowired
    ChatWithoutRag chatWithoutRag;

    @Autowired
    ChatClient agentAsAJudge;

    @Test
    void promptWithoutRagShouldKnowSpringBoot() {
        var response = chatWithoutRag.promptWithoutRag( question: "What is Spring Boot?");

        assertThat(response).isNotEmpty();
        assertThat(response).contains("Java", "framework");

        //LLM as a judge
        var promptTemplate = PromptTemplate.builder().template("Is the answer related to Java? {ANSWER}")
                .variables(Map.of( k1: "ANSWER", response)).build();

        agentAsAJudge.prompt(promptTemplate.create())
                .options(ChatOptions.builder().model("claude-haiku-5-5"));
    }
```

Building Agentic Workflows

Agentic Workflows

https://www.anthropic.com/engineering/building-effective-agents

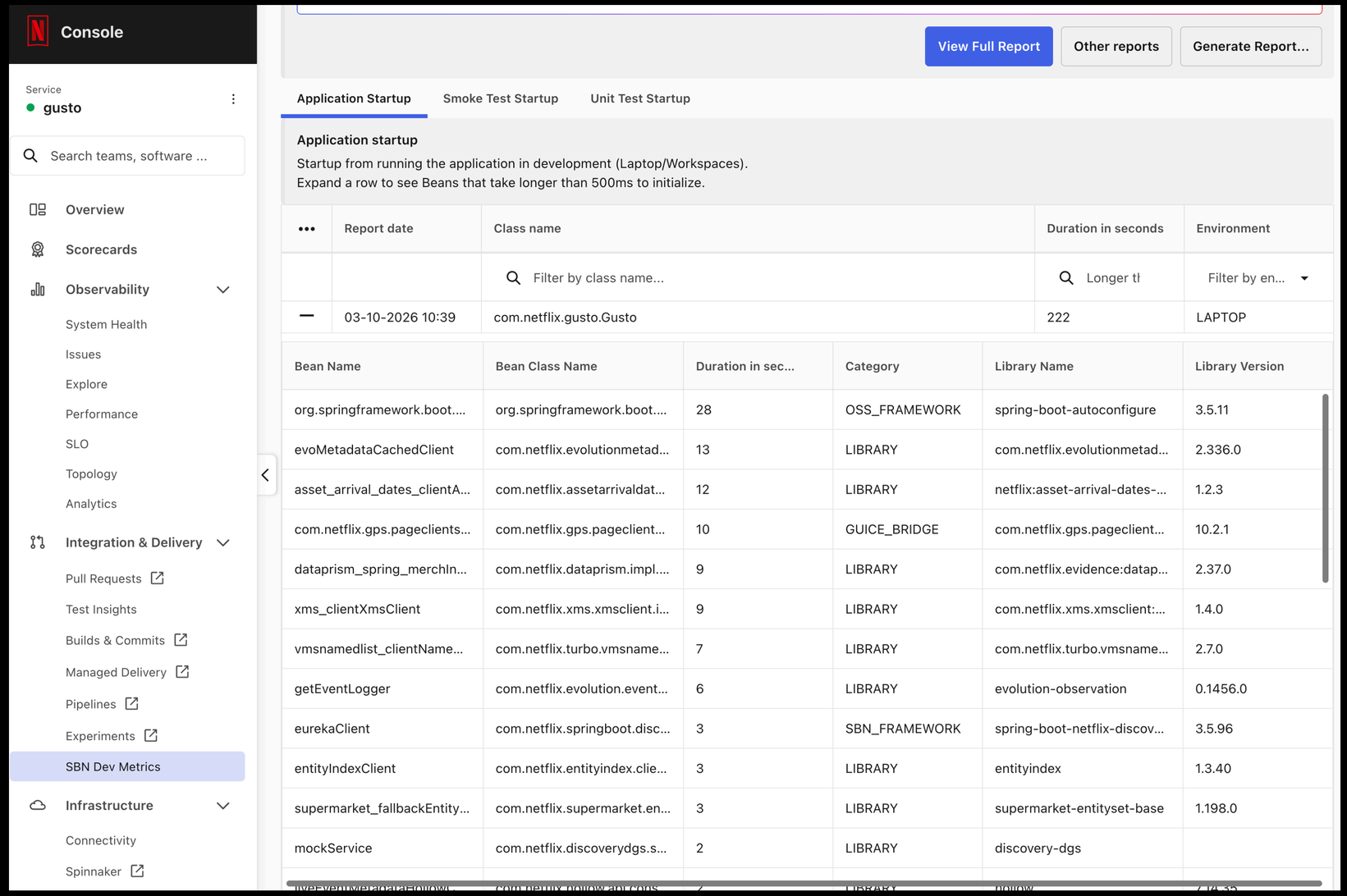
- You keep control of the workflow
- Use LLM in parts of the workflow
- Adding AI while keeping control and determinism



Real world example: SBN Dev Metrics



Real world example: SBN Dev Metrics



Console

Service
gusto

Console / Org / Member Experience ... / Software & ... / S... / g.. / SBN Dev...

SBN Dev Metrics: gusto Alpha

SBN Dev metrics give insights into the startup of the application during development and running tests

Startup Analysis Report

loading in library beans.

Top Recurring Bean Bottlenecks (Application Starts)

The following beans appear consistently across all application startup reports as the most time-consuming. They are ranked by their recurring impact.

🔴 1. `evoMetadataCachedClient` — Evolution Metadata Client

- **Library:** `com.netflix.evolutionmetadata:evolution-metadata-client` v2.336.0
- **Category:** LIBRARY (Netflix internal)
- **Typical duration:** 12s–58s (outlier: 197s)
- **Appears in:** Every application report as top-1 or top-3

What it does: This bean initializes `EvoMetadataCachedClient`, which calls `this.gutenbergSubscriber.startSubscriber()` **synchronously in the constructor**. This subscriber connects to Gutenberg (Netflix's internal data platform), subscribes to metadata topics, and **downloads and caches the entire workflow metadata corpus into memory** before the constructor returns. The class has a `waitForMetadataReady(int maxWaitSeconds)` method, confirming this is a known slow operation. The blocking nature means the entire startup is held up while remote metadata is fetched, deserialized, and indexed. Network latency, dataset size, and remote service responsiveness all contribute to the high variance (12s to 197s+).

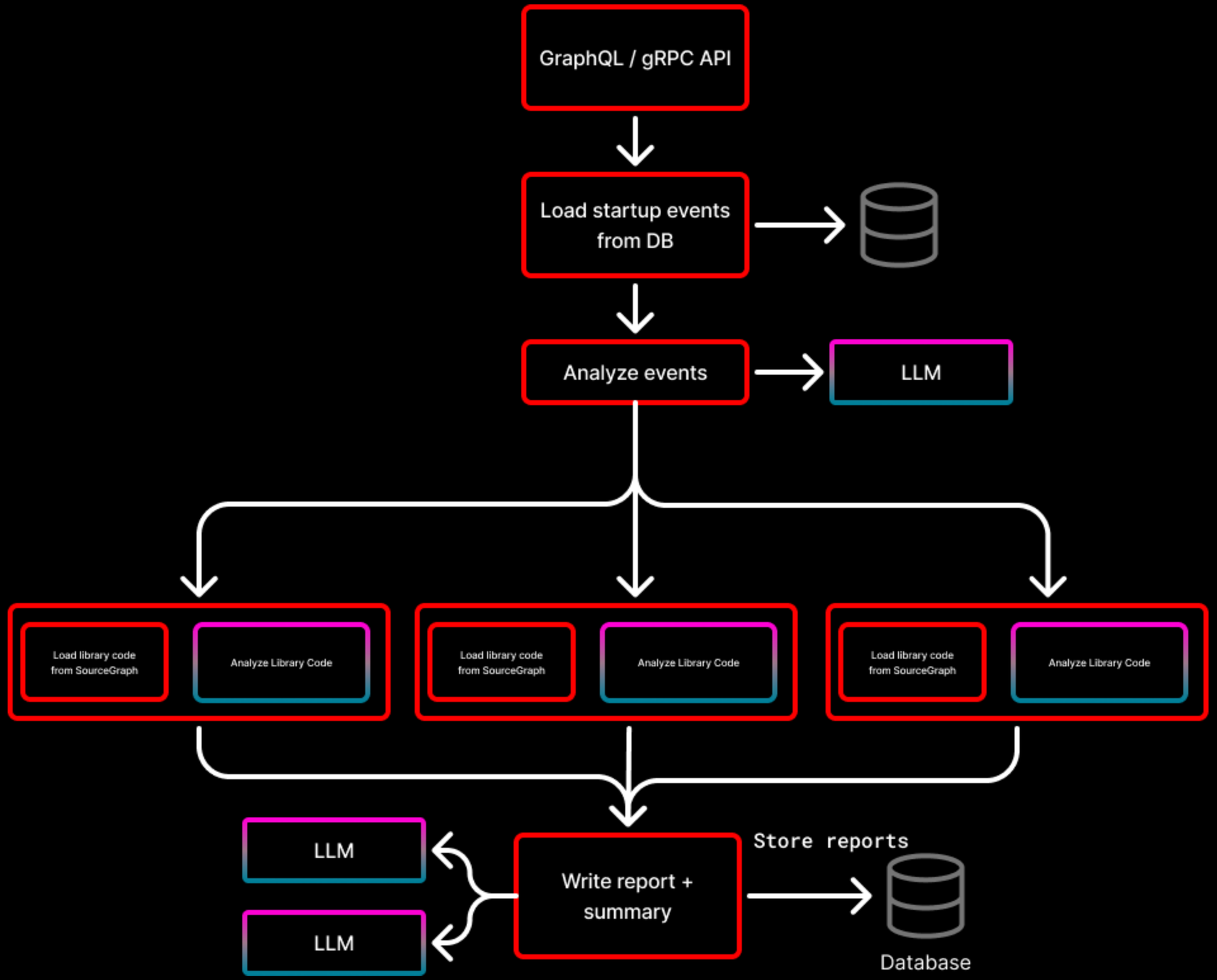
🔴 2. `asset_arrival_dates_clientAssetArrivalDatesClientFactoryBean` — Asset Arrival Dates Client

- **Library:** `netflix:asset-arrival-dates-client` v1.2.3
- **Category:** LIBRARY (Netflix internal)
- **Typical duration:** 13s–88s (outlier: 465s)
- **Appears in:** Every application report as top-1 or top-3

What it does: The constructor of `AssetArrivalDatesClientImpl` calls `.buildAsync().join()` on a Netflix Hollow/Cinder consumer, which **blocks the calling thread** until the entire Asset Arrival Dates Hollow dataset is downloaded from Cinder, deserialized, and loaded into memory. After that, it builds a `UniqueKeyIndex` over all records keyed on `videoId`. For a catalog the size of Netflix's (potentially millions of video/country/language combinations), this involves substantial network I/O, memory allocation, and index construction — all synchronously during bean creation.

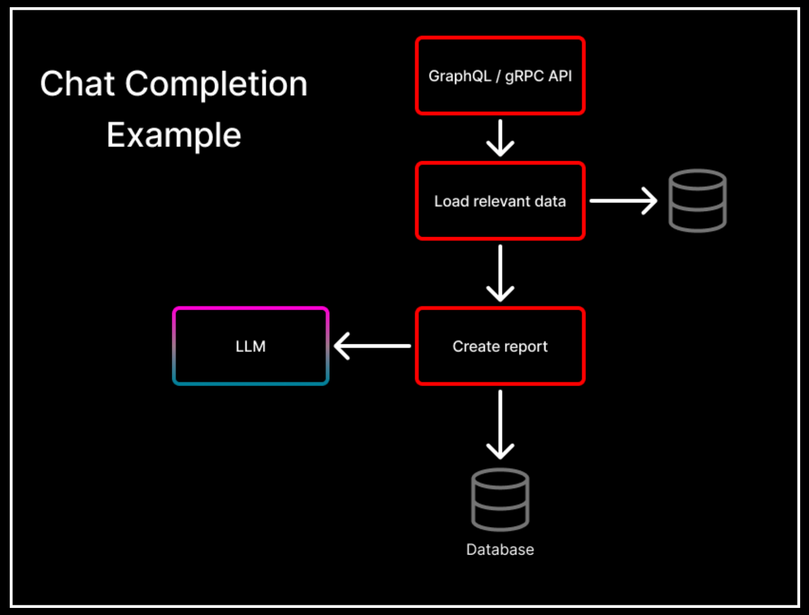
SBN Dev Metrics	+	02-12-2026 14:43	com.netflix.gusto.Gusto	575	LAPTOP

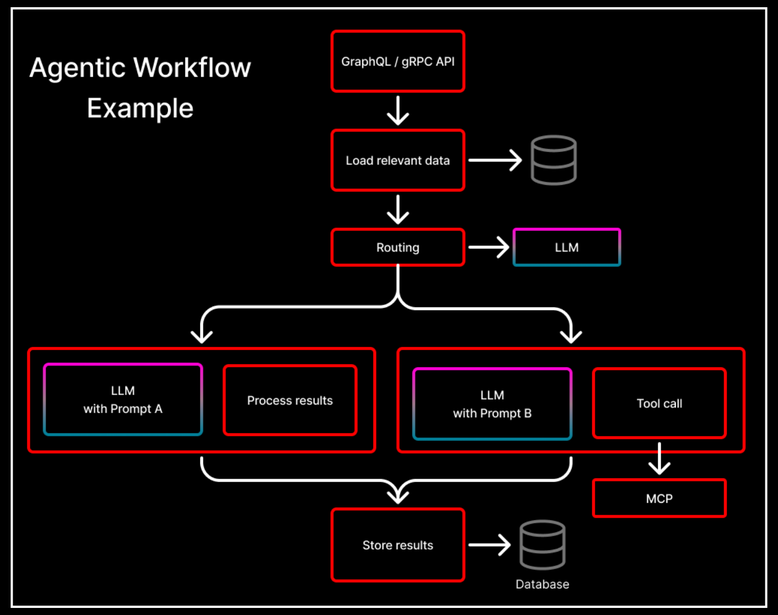
@pbakker

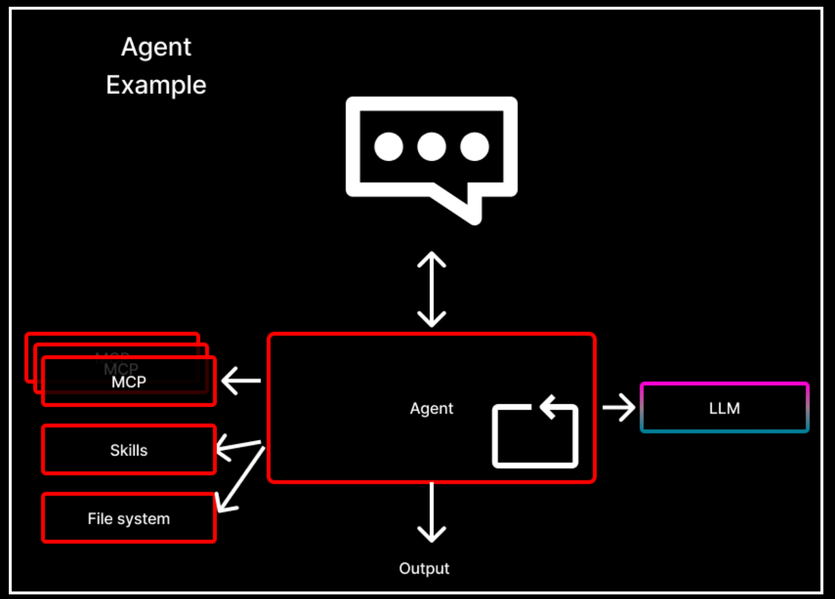

GraphQL / gRPC API
Load startup events from DB
Analyze events
LLM
Load library code from SourceGraph
Analyze Library Code
Load library code from SourceGraph
Analyze Library Code
Load library code from SourceGraph
Analyze Library Code
LLM
LLM
Write report + summary
Store reports
Database

Managed Agents

4

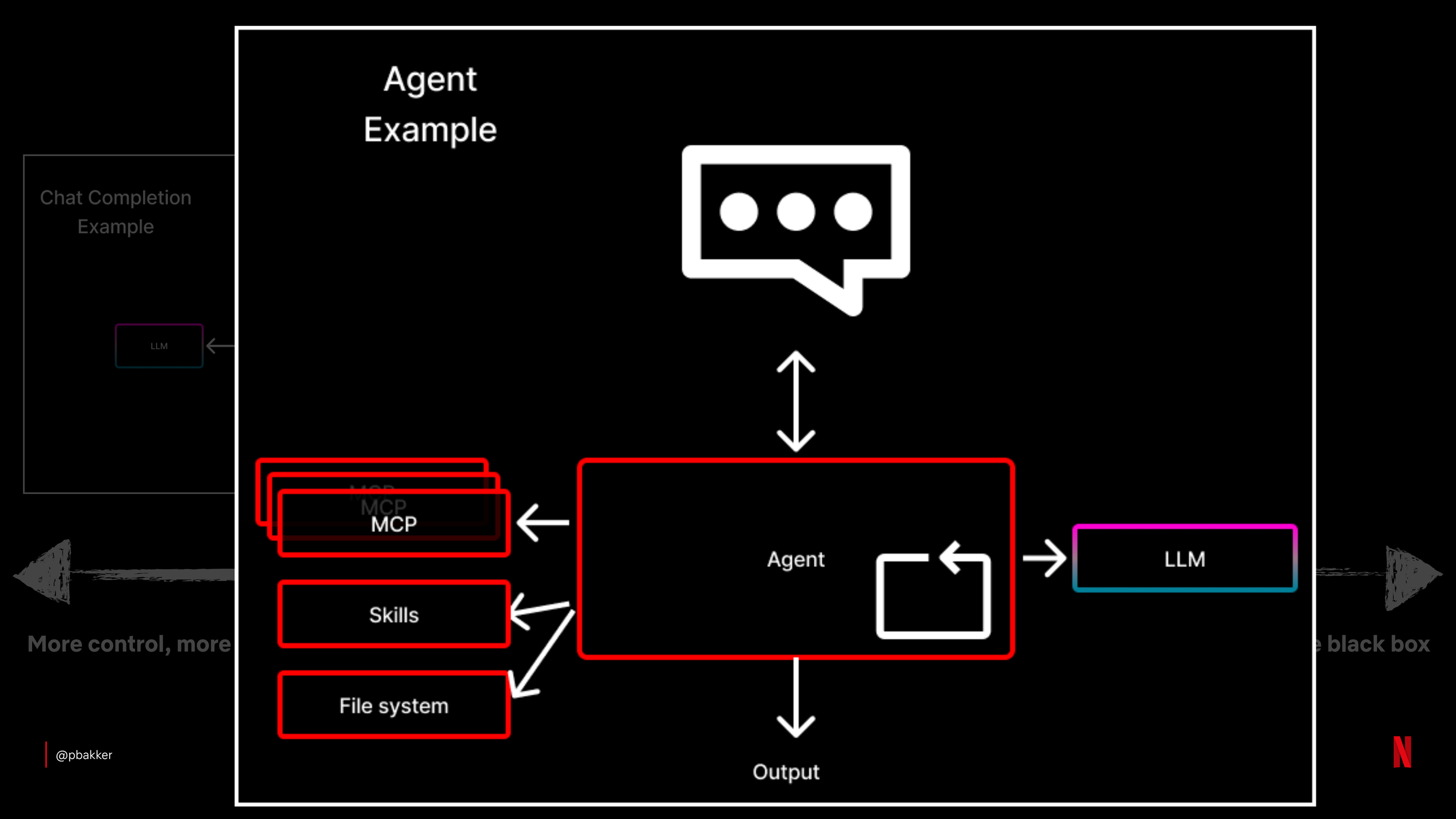






More control, more transparency

More agentic, more black box


Chat Completion
Example
LLM
Agent
Example
MCP
MCP
Skills
File system
Agent
LLM
Output
More control, more
black box
@pbakker

Managed Agent APIs

- Anthropic Managed Agent API
- OpenAI Agents API
- 1st party Agents APIs